Priority-Aware Grid Sensing for Scalable Heterogeneous Multi-Robot Coordination

1st Rehab Uddin Shawon
Department of Computer Science
Missouri State University
Springfield, MO, USA
rs424s@MissouriState.edu

2nd Siming Liu
Department of Computer Science
Missouri State University
Springfield, MO, USA
SimingLiu@MissouriState.edu

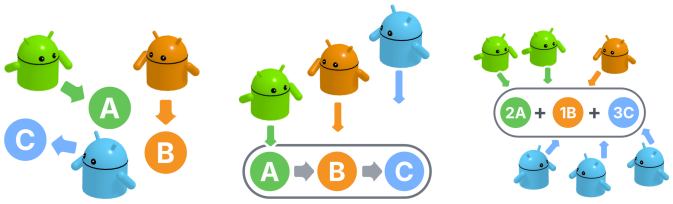
***Abstract*—Coordinating large-scale heterogeneous robot teams remains a significant challenge, especially as the number of robots and task types increases. Existing approaches often degrade as task diversity grows, limiting their scalability. In this paper, we propose a shared-policy Multi-Agent Reinforcement Learning (MARL) framework for heterogeneous multi-robot coordination using priority-aware grid sensing and type-aware observation encoding. The framework combines one-hot agent type representations with a customized grid sensor that embeds task relevance directly into the observation space, enabling robots to identify and prioritize suitable tasks while sharing a single neural policy. This design allows heterogeneous robots to learn task-specialized behaviors in environments with up to ten robot and task types, substantially improving scalability over our previous approach. We further introduce generalized formulations for sequential and collaborative tasks. The sequential formulation supports task sequences of arbitrary length and order, while the collaborative formulation enables dynamic participation by varying numbers of heterogeneous robots without predefined execution orders. Experiments in Unity platform across 3-type, 5-type, and 10-type scenarios show consistent learning, improved scalability over Unity's default grid sensor, and strong generalization to unseen task sequences and collaboration requirements. These results demonstrate that the proposed observation design provides an effective and scalable solution for heterogeneous MARL in complex multi-robot environments.**

***Index Terms*—MARL, Heterogeneous Multi-Agent Systems, Priority-Aware Grid Sensing, Multi-Robot Coordination.**

I. Introduction

Multi-Agent Reinforcement Learning (MARL) enables autonomous agents to learn cooperative behaviors through interaction with dynamic environments [1], with recent success in robotics, task allocation, and coordination [2]. Heterogeneous systems, where agents differ in role or capability, are especially relevant to warehouse automation, disaster response, and service robotics [3]. Despite this progress, scaling heterogeneous MARL remains difficult: most approaches target small numbers of agent types, use separate per-type policies, or assume fixed task structures [4], [5], causing unstable training as type count grows and leaving limited flexibility for dynamic sequential or collaborative workflows.

Fig. 1 illustrates the three task categories considered in this work: heterogeneous, sequential, and collaborative tasks. Our previous work [6] successfully coordinated heterogeneous robots in environments with two and three robot types, but performance deteriorated beyond three types, limiting applicability to larger systems. We address this with a scalable shared-policy MARL framework built on a priority-aware grid sensor that, unlike conventional grid sensors encoding only object presence, embeds task relevance directly into the observation. Combined with one-hot type encoding, this lets robots distinguish relevant from irrelevant tasks and learn specialized behaviors at substantially larger N. We further generalize sequential tasks to arbitrary order/length and collaborative tasks to dynamic requirement vectors specifying remaining needs without predefined arrival order, enabling flexible coordination with policies that generalize to unseen sequences and requirements. We evaluate the framework in Unity across three, five, and ten robot types, comparing against Unity's default grid sensor under identical training conditions. The main contributions of this work are as follows:



(a) Heterogeneous Tasks (b) Sequential Tasks (c) Collaborative Tasks

Fig. 1. Heterogeneous robots performing multiple task types.

- We proposed a scalable shared-policy MARL framework that enables heterogeneous robots to learn specialized behaviors across up to ten robot and task types.
- We introduced a priority-aware grid sensing mechanism that incorporates task relevance information into the observation space, significantly improving learning scalability compared with conventional grid-based sensing.
- We developed generalized formulations for sequential and collaborative tasks that support arbitrary task sequences, dynamic participation, and flexible coordination.
- We provided extensive experimental evaluations demonstrating improved scalability, adaptability, and generalization across heterogeneous, sequential, and collaborative multi-robot coordination tasks.

II. Related Work

Research on MARL has expanded rapidly, particularly in its application to heterogeneous robotic systems, transfer learning, and scalable multi-robot coordination.

A. Scalable Identity Encoding for Heterogeneous MARL

Early heterogeneous frameworks relied on ontology-based agent mappings [7], while recent shared-policy approaches embed identity directly in the observation [8], most commonly via one-hot encoding (OHE), valued for its simplicity and natural extension to arbitrary N [9]. Dense alternatives such as our prior Signed Type Encoding [6] improved gradient propagation for small N (two, three) over OHE, but as N grows, the dimensionality and conditioning of dense ± 1 encodings become harder to manage alongside an increasingly complex observation space. Heterogeneous-agent Markov game [4] and role-decomposition [10] formulations ground shared representations theoretically but typically use a small, fixed class count. We revisit OHE for its scalability, since each type occupies an independent dimension with no growing cross-type interaction, and show that, paired with an informative perception module, it supports stable learning even at $N = 10$.

B. Sensor-Based Perception for Multi-Agent Coordination

Many MARL studies use grid or ray-cast sensors [11] encoding per-cell occupancy of predefined layers (agents, obstacles, goals), but provide no agent-specific signal about a detected object's *relevance* to the agent's goal. Attention-based [12], [13], graph-based [14], and awareness-map-based [15] methods show relevance-aware representations improve coordination, but add network modules rather than modifying the sensor itself. Explainability techniques for MARL [16] augment observations with context but do not address scaling spatial sensing with object-type count. We address this by designing a priority-aware grid sensor that retains the per-layer occupancy encoding but adds a single *priority block* per cell, a robot-specific, distance-weighted relevance signal, which we show is critical for scaling to ten robot types.

C. Generalized Sequential and Collaborative Coordination

Cooperative task execution has been studied via counterfactual credit assignment [17] and value decomposition [18]. Task-allocation frameworks [19] typically assume a fixed, known task structure, while collision-avoidance approaches address motion rather than task structure. Transfer-learning and curriculum approach [20] shows generalization across related task structures, but not across *arbitrary* sequence orderings or collaborative requirements at scale. We generalize both: a sequential task's identity transitions through an arbitrary ordered label list, and a collaborative task carries a dynamic requirement vector updated as robots of any type arrive in any order.

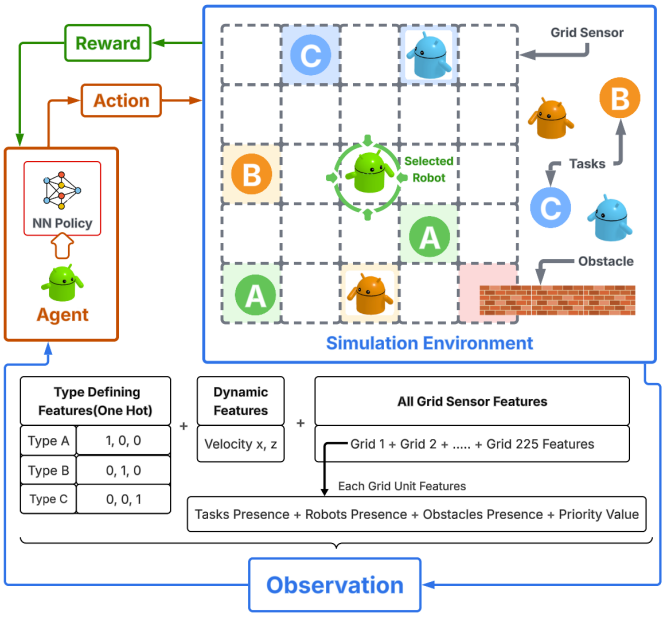


Fig. 2. Heterogeneous Multi-Agent Reinforcement Learning Framework

III. Methodology

This section presents the proposed framework. Unlike our prior approach, limited to few robots and fixed task structures, it introduces a priority-aware observation mechanism supporting larger N and generalized sequential/collaborative execution. Fig. 2 provides an overview of the framework, illustrating the custom grid sensor, observation construction, and shared policy network described in this section.

A. Problem Formulation

We model heterogeneous robot systems as a Markov game, extending the MDP framework to multi-agent settings:

$$G = \langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}, \{\mathcal{R}_i\}, \mathcal{P}, \gamma \rangle, \tag{1}$$

where $\mathcal{N}$, $\mathcal{S}$, $\{\mathcal{A}_i\}$, $\{\mathcal{R}_i\}$, $\mathcal{P}$, and γ denote the agent set, state space, per-agent action spaces, reward functions, transition probability, and discount factor. At each timestep, agent i observes $o_i \in \mathcal{O}_i$, selects an action $a_i \in \mathcal{A}_i$ based on a policy $\pi_\theta(a_i \mid o_i)$, and receives a reward:

$$r_i = \mathcal{R}_i(s, \mathbf{a}, s'), \qquad \mathbf{a} = \{a_0, a_1, \ldots, a_{N-1}\}, \tag{2}$$

where $\mathbf{a}$ denotes the joint action of all agents and s' is the resulting next state. The policy π_θ is trained using a policy optimization algorithm suitable for decentralized execution, which facilitates coordinated learning among the agents.

Heterogeneity spans N robot types $\{R_0, \ldots, R_{N-1}\}$ and N task types $\{T_0, \ldots, T_{N-1}\}$: R_k is rewarded for completing type-T_k tasks and penalized otherwise. All task types are present simultaneously and any robot may interact with any task, but only own-type completions are rewarded, enforcing type-specific specialization and requiring robots to distinguish relevant from irrelevant tasks. A robot is considered to *interact with* a task when it enters the task's designated collision region

in the environment; this interaction triggers task-completion logic (Sections III-B and III-C) and the corresponding reward. As in [6], all types share a policy network π_θ, with type-specific behavior emerging from the observation vector alone. Unlike prior work ($N \in \{2, 3\}$), we target $N \in \{3, 5, 10\}$, requiring an observation design that scales gracefully.

B. Priority-Aware Grid Sensor and Observation Design

The observation vector provided to each robot consists of three components:

$$O = [\, O_{type},\ O_{dyn},\ O_{grid}\,], \tag{3}$$

where O_{type} encodes robot identity, O_{dyn} encodes dynamic robot features (defined below), and O_{grid} encodes spatial information produced by the proposed custom grid sensor, which is the central contribution of this framework.

1) Type and Dynamic Features: Robot identity is represented using one-hot encoding (OHE). For an N-type environment, the encoding vector for robot type k is

$$\mathbf{e}_k \in \{0,1\}^N, \qquad e_k(j) = \begin{cases} 1, & j = k \\ 0, & j \neq k, \end{cases} \tag{4}$$

so that only the kth position is active. For example, in a five-type environment, $R_A = [1, 0, 0, 0, 0]$, $R_B = [0, 1, 0, 0, 0]$, and so on. The dynamic feature vector O_{dyn} consists of the robot's current 2D velocity in the world frame,

$$O_{dyn} = [v_x, v_y], \tag{5}$$

which allows the policy to condition its action on the robot's current motion state in addition to its static type identity.

2) Limitations of a Naive Grid Sensor at Scale: A naive grid sensor discretizes the surrounding area into cells and encodes per-cell object presence as binary indicators across predefined layers. This has two drawbacks at scale: the per-cell feature size grows linearly ($2N + 1$ dimensions), and, more critically, the sensor gives *no relevance signal*, own-type and irrelevant-type task cells differ only in which equally weighted dimension is set to 1, forcing the policy to infer relevance from sparse co-occurrence statistics, a harder credit-assignment problem as N grows.

3) Extended Grid Sensor Design: To address this limitation, we extended the grid sensor that retains the per-layer occupancy encoding but augments each cell with an additional scalar *priority block* that explicitly encodes task relevance and proximity. The sensing area is discretized into a fixed 15×15 grid of 2×2 unit cells, covering a 30×30 unit footprint centered on the robot. For an N-type environment, each cell senses N task-type layers, N robot-type layers, one obstacle layer, and one priority value, giving $2N + 2$ values per cell. For grid cell c, the occupancy of the cell feature vector is

$$\mathbf{f}_c^{occ} = [\, \underbrace{x_0, \ldots, x_{N-1}}_{\text{task layers}},\ \underbrace{y_0, \ldots, y_{N-1}}_{\text{robot layers}},\ z\,] \tag{6}$$

where $x_j, y_j, z \in \{0, 1\}$ indicate the presence of a task of type T_j, a robot of type R_j and an obstacle respectively in cell c.

The priority value p_c for cell c, as observed by a robot of type R_k, is initialized to $p_c = -1$ and is recomputed as:

$$p_c = \begin{cases} 10 \cdot w(d_c) & \text{if } c \text{ contains a task of type } T_k \\ -1 & \text{otherwise} \end{cases} \tag{7}$$

where d_c is the distance from the robot to cell c, and $w(d_c)$ is a monotonically decreasing weighting function of distance (e.g., $w(d_c) = 1/d_c$) that assigns larger priority magnitudes to closer cells. The full cell feature vector is:

$$\mathbf{f}_c = [\mathbf{f}_c^{occ},\ p_c] \in \mathbb{R}^{2N+2} \tag{8}$$

The priority block serves two purposes. First, it gives a dense, per-cell relevance signal, removing the need for the policy to infer relevance from sparse co-occurrence statistics alone. Second, when a robot's own-type task appears in multiple cells, the distance-weighted priority values provide a built-in preference ordering for which task instance to approach first, allowing robots to distinguish not only which tasks are relevant but also which relevant task should be prioritized.

The complete grid observation is obtained by concatenating all grid-cell feature vectors:

$$O_{grid} = [\mathbf{f}_0, \mathbf{f}_1, \ldots, \mathbf{f}_{n-1}], \qquad n = 225 \tag{9}$$

corresponding to the $15 \times 15 = 225$ cells of the grid.

4) Full Observation Vector: The complete observation for a robot of type R_k in an N-type scenario concatenates the three components introduced above:

$$o_k = [\ \underbrace{\mathbf{e}_k}_{\text{type encoding}},\ \underbrace{(v_x, v_y)}_{\text{velocity}},\ \underbrace{\mathbf{f}_1, \mathbf{f}_2, \ldots, \mathbf{f}_{225}}_{\text{grid sensor } (15\times 15 \text{ cells})}\ ] \tag{10}$$

The total observation dimensionality is $N+2+225\cdot(2N+2)$, which for $N = 3, 5, 10$ corresponds to 1805, 2707, and 4962 dimensions, respectively. Fig. 2 illustrates the overall framework, including the custom grid sensor, observation construction, and shared policy network.

5) Arbitrary Sequence Encoding: In our prior work [6], a sequential task was modeled as a single fixed-length, fixed-order chain (e.g., In a three-type environment $T_A \rightarrow T_B \rightarrow T_C$), where each step could only be performed by a single, predetermined robot type, and the total sequence length was bounded by N. This precludes sequences that revisit a task type, omit a task type, or exceed length N, all of which arise naturally in real world workflows. In this study, we generalize sequential tasks by representing each sequential task instance as an ordered list of task-type labels of arbitrary length and composition, drawn from $\{T_1, \ldots, T_N\}$ with repetition and arbitrary ordering permitted:

$$Seq = \langle T_{j_0}, \ldots, T_{j_{L-1}} \rangle, \quad j_l \in \{0, \ldots, N-1\},\ L \geq 1 \tag{11}$$

For example, $Seq = \langle T_A, T_C, T_A, T_B, T_B, T_C \rangle$ is a valid six-step sequence. Each instance maintains a step pointer l (initialized to 1) and presents as type T_{j_l}; a correct interaction by robot R_{j_l} advances $l \leftarrow l + 1$ and updates the task's identity, until $l > L$ marks it complete. This supports arbitrary sequence lengths and orderings while reusing the same per-type behaviors.

6) Generalization Across Sequences at Evaluation Time: Because the task-progression mechanism and the underlying per-step recognition behavior are identical to those used for heterogeneous (single-step) tasks, a policy trained on a sequence covering all N task types, e.g., the increasing sequence $\langle T_A, T_B, \ldots \rangle$, learns the general skill of recognizing the current task type and responding with the corresponding robot type, rather than memorizing a specific order. For training, we use $Seq_{train} = \langle T_A, T_B, T_C \rangle$, $\langle T_A, \ldots, T_E \rangle$, and $\langle T_A, \ldots, T_J \rangle$ for the 3-, 5-, and 10-type configurations, respectively, and at evaluation time we test on alternative sequences (e.g., $\langle T_B, T_C, T_A \rangle$ or $\langle T_A, T_E, T_B, T_C, T_A \rangle$) composed of the same underlying types.

C. Generalized Collaborative Task Coordination

1) Dynamic Task-Requirement Encoding: In [6], a collaborative task required exactly one robot of each of the N types to arrive in a strict predetermined order, completing only once all N robots had arrived in that order (e.g., In a three-type environment $R_A \rightarrow R_B \rightarrow R_C$). This restricts collaborative tasks to at most N robots, exactly one per type, in a single fixed order, none of which reflect realistic scenarios such as heavy-lifting tasks requiring multiple robots of the same type, or robots arriving in whatever order they happen to finish their previous activity. We address this by associating each collaborative task instance with a *requirement vector*

$$\mathbf{c} = [c_0, c_1, \ldots, c_{N-1}] \in \mathbb{Z}^N_{\geq 0}, \tag{12}$$

where c_j denotes the number of robots of type R_j still required for task completion. For example, in a five-type scenario, $\mathbf{c} = [3, 1, 2, 0, 0]$ indicates that there are 5 types of tasks present in the environment and the task still requires three robots of type R_A, one of R_B, and two of R_C.

When robot R_j interacts with a collaborative task, if $c_j > 0$ it becomes *engaged* (waits at the task, unavailable elsewhere) and $c_j \leftarrow c_j - 1$; if $c_j = 0$, the interaction has no effect and the robot remains free. The task completes when $\mathbf{c} = \mathbf{0}$, releasing all engaged robots. Unlike prior work, no arrival order is predefined, any required type may arrive first, and a type may appear multiple times in $\mathbf{c}$.

2) Reflecting Task Requirements in the Grid Sensor: To let nearby robots perceive which types a task currently needs, the requirement vector $\mathbf{c}$ is reflected into the task-layer occupancy of the grid sensor for any cell containing the task:

$$x_j = \begin{cases} 1, & c_j > 0 \\ 0, & c_j = 0 \end{cases} \qquad j = 0, \ldots, N-1 \tag{13}$$

As robots become engaged and requirements are fulfilled, this occupancy pattern updates dynamically, mirroring changes in $\mathbf{c}$ and providing real-time guidance regarding which robot types are still needed. Combined with the priority block, a robot of type R_j approaching a collaborative task perceives a positive priority value for that cell only while $c_j > 0$, naturally directing robots toward collaborative tasks that still require their type.

TABLE I. COLLABORATIVE TASK REQUIREMENT TEMPLATES USED FOR TRAINING

Config	Task	Requirement Vector $\mathbf{c}$	Grid Sensor Observation
3-Type	1	$[2, 1, 0]$	$[1, 1, 0]$
	2	$[0, 1, 2]$	$[0, 1, 1]$
5-Type	1	$[3, 1, 1, 0, 0]$	$[1, 1, 1, 0, 0]$
	2	$[0, 1, 3, 1, 0]$	$[0, 1, 1, 1, 0]$
	3	$[0, 0, 1, 1, 3]$	$[0, 0, 1, 1, 1]$
10-Type	1	$[2, 2, 2, 1, 0, 0, 0, 0, 0, 0]$	$[1, 1, 1, 1, 0, 0, 0, 0, 0, 0]$
	2	$[0, 0, 0, 1, 2, 2, 1, 0, 0, 0]$	$[0, 0, 0, 1, 1, 1, 1, 0, 0, 0]$
	3	$[0, 0, 0, 0, 0, 0, 1, 2, 2, 2]$	$[0, 0, 0, 0, 0, 0, 1, 1, 1, 1]$

3) Training Compositions: To give all robot types balanced opportunities to learn collaborative behavior, we design multiple collaborative task templates per configuration whose requirement vectors collectively cover all N types in equal proportion (Table I). After training, we evaluate whether the learned policy generalizes to collaborative tasks with *different* requirement vectors than those seen during training.

D. Reward Function Design

We adopt a reward structure that is consistent across the heterogeneous and sequential settings, with a separate structure for collaborative tasks. In all settings, the total reward is

$$\mathcal{R} = R_{task} + R_{move} + R_{collision} + R_{time} \tag{14}$$

The task reward rewards correct task selection and penalizes incorrect interactions:

$$R_{task} = \begin{cases} +10, & \text{correct task} \\ -1, & \text{wrong task} \end{cases} \tag{15}$$

For heterogeneous/sequential tasks, a robot is penalized per step for moving slowly, encouraging continuous exploration:

$$R_{move} = \begin{cases} -0.01, & \|v\| < 1.5 \\ 0, & \text{otherwise} \end{cases} \tag{16}$$

and incurs a fixed penalty for colliding with an obstacle or with another robot: $R_{collision} = -0.1$. A small per-step time penalty is applied throughout to encourage efficient task completion: $R_{time} = -0.001$.

For collaborative tasks, the low-velocity and robot-robot collision penalties are removed so waiting robots are not penalized, while obstacle-collision and time penalties are retained. This unified structure, simpler than the asymmetric per-type rewards used in [6], is enabled by the dynamic requirement-vector mechanism that guides robots toward tasks requiring their capabilities through the priority and occupancy information encoded in the grid sensor.

E. Training Procedure

All experiments were conducted in Unity game engine environment, with the same network shared across all robot types and task categories. We evaluate the framework across three independent task settings:

1) Heterogeneous task learning, in which robots learn to recognize and approach tasks corresponding to their

TABLE II. TRAINING PERFORMANCE COMPARISON BETWEEN THE PROPOSED METHOD AND BASELINE ACROSS DIFFERENT TASK SETTINGS.

	Heterogeneous		Sequential		Collaborative	
Metric	Prop.	Base.	Prop.	Base.	Prop.	Base.
			3-Type Tasks			
Cum. Rwd.	**812.4**	558.6	**720.2**	218.3	**413.2**	79.7
Std. Dev.	**8.7**	11.3	**10.4**	43.55	**4.9**	10.76
95% CI	**7.3**	10.5	**9.6**	45.7	**4.6**	11.3
			5-Type Tasks			
Cum. Rwd.	**590.1**	137.7	**521.3**	-4.5	**205.6**	4.08
Std. Dev.	**7.9**	17.9	3.5	**1.3**	20.37	**2.15**
95% CI	**7.3**	16.5	3.2	**1.2**	25.3	**2.2**
			10-Type Tasks			
Cum. Rwd.	**445.1**	-5.4	**366.1**	-5.2	**273.5**	-0.4
Std. Dev.	131.3	**0.56**	6.3	**0.3**	8.31	**1.07**
95% CI	121.4	**0.5**	10.1	**0.25**	8.7	**0.9**

TABLE III. ROBOT/TASK DISTRIBUTION

Configuration	Per-Type Count	Total Robots	Total Tasks
3-Type	13, 13, 14	40	40
5-Type	8 each	40	40
10-Type	4 each	40	40

own type using the proposed priority-aware observation framework.

2) Sequential task learning, in which robots learn to execute generalized sequential workflows by responding to dynamically changing task-type requirements.
3) Collaborative task learning, in which robots learn to satisfy dynamic collaborative-task requirements and coordinate with other robots through the proposed requirement-vector mechanism.

All three task categories are trained independently from scratch. The priority-aware observation design provides a sufficiently informative and consistent representation across task types that each stage learns effectively on its own, without warm-starting from another.

IV. EXPERIMENTAL SETUP

All experiments were conducted in the Unity Game Engine [11], building on a multi-agent environment as in our prior work [6]. The environment is a 40×40 unit square arena enclosed by walls, which also serve as obstacles.

A. Simulation Environment and Configurations

Total robots and tasks are fixed at 40 across all configurations, distributed evenly across N types ($13, 13, 14$ for 3-type; 8 and 4 per type for 5- and 10-type; Table III), so performance differences reflect N rather than scale or density. Each configuration is trained on all three task settings (heterogeneous, sequential, collaborative) using the proposed sensor with one-hot encoding, plus a baseline using Unity ML-Agents' default grid sensor. All settings train from scratch for 10 million steps (5,000-step episodes), with seven seeded runs per (configuration $\times$ task $\times$ sensor) combination, 126 runs total. Results are reported as means across seven runs with standard deviation and 95% confidence interval (Table II).

B. Neural Network Configuration

Robots are trained with PPO in Unity. The discrete action space consists of the four cardinal movement directions plus a no-op. The observation vector, of dimensionality $N + 2 + 225(2N + 2)$ (Section III-B), is fed directly into a fully connected feedforward network. The network consists of a single hidden layer of 256 units with ReLU activation, and produces two outputs: a policy output, a fully connected layer producing a discrete action distribution over the movement actions, and a value output, a single fully connected unit estimating the state value required for PPO optimization. This architecture is shared across all robot types and all three configurations ($N = 3, 5, 10$), with only the input dimensionality changing as N scales (1805, 2707, and 4962 dimensions, respectively, as noted in Section III-B).

C. Evaluation Metrics

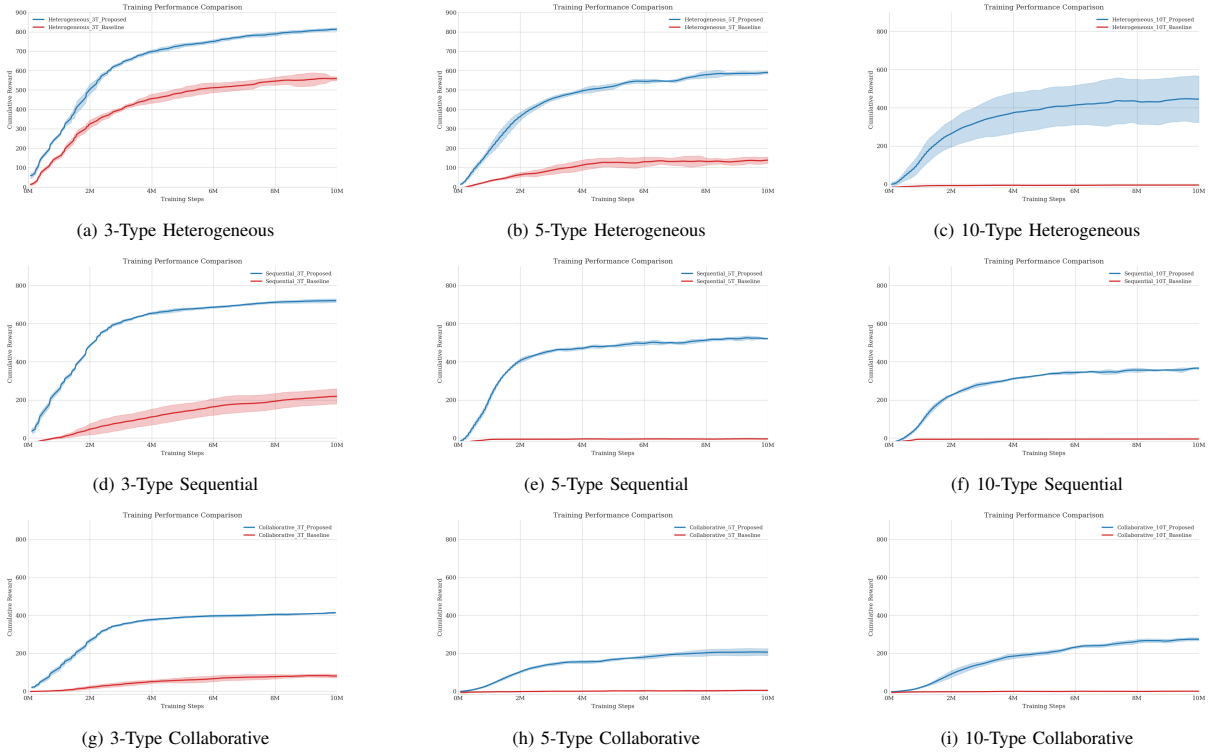
To evaluate the proposed approach, we compared trained robots across different configurations and task types against a baseline using Unity ML-Agents' built-in grid sensor. Both approaches used identical training and environment settings for a fair comparison. The best-performing model from each configuration was evaluated for 1000 steps using task-level metrics measuring task completion, efficiency, and multi-agent coordination. The evaluation metrics are defined as follows:

- **Cumulative Reward:** Total reward accumulated by all robots over 1000 evaluation steps, reflecting overall policy effectiveness and training stability.
- **Correct Task Interactions:** The number of times a robot completes a task that belongs to its own type within 1000 steps. This metric evaluates the quality of specialization and identifies desired cross-task interactions.
- **Incorrect Task Interactions:** The number of times a robot worked on a wrong task within 1000 steps. This metric evaluates the quality of specialization and identifies undesired cross-task interactions.
- **Task Completion Rate:** Total tasks completed within 1000 steps, measuring robots' ability to accomplish assigned work.
- **Task Interaction Delay:** The average number of steps each task needs to wait after spawning before getting completed by a robot. Lower values indicate greater efficiency and faster task execution.
- **Coordination Delay:** The amount of time robots waited before completing tasks. Lower delay indicates better cooperation and synchronization among robots.

Together, these metrics provide a comprehensive evaluation of task performance, specialization effectiveness, and collaborative efficiency in multi-agent settings.

V. RESULTS AND DISCUSSION

This section presents training and evaluation results for the proposed priority-aware grid sensor with one-hot type encoding, compared against the default-grid-sensor baseline, across 3-, 5-, and 10-type configurations and heterogeneous, sequential, and collaborative task settings. In all training



(a) 3-Type Heterogeneous (b) 5-Type Heterogeneous (c) 10-Type Heterogeneous

(d) 3-Type Sequential (e) 5-Type Sequential (f) 10-Type Sequential

(g) 3-Type Collaborative (h) 5-Type Collaborative (i) 10-Type Collaborative

Fig. 3. Comparison of training performance between the proposed approach and the baseline approach across heterogeneous, sequential, and collaborative task settings under 3-type, 5-type, and 10-type configurations.

TABLE IV. OVERALL PERFORMANCE SUMMARY ACROSS TASK SETTINGS AND CONFIGURATIONS (CUSTOM GRID SENSOR, PER 1000 STEPS)

Set.	Con.	Mthd.	Cor. T.	Incor. T.	Cum. R.	Tot. T.	Int. Del.	Crd. Del.
Het.	3T	Prop.	**815**	**222**	**7928**	**815**	1.51	–
		Base.	374	763	2977	374	**1.12**	–
	5T	Prop.	**576**	**212**	**5548**	**576**	1.64	–
		Base.	152	637	883	152	**1.51**	–
	10T	Prop.	**402**	203	**3817**	**402**	**1.95**	–
		Base.	1	**8**	2	1	5.12	–
Seq.	3T	Prop.	**806**	**330**	**7730**	**255**	**4.6**	–
		Base.	229	609	1681	64	9.3	–
	5T	Prop.	**621**	349	**5861**	**108**	**8.77**	–
		Base.	4	**10**	30	0	30.3	–
	10T	Prop.	**481**	370	**4440**	**31**	**17.0**	–
		Base.	1	**2**	8	0	30.3	–
Col.	3T	Prop.	**358**	**60**	**3500**	**116**	**2.9**	**1.23**
		Base.	115	171	979	33	6.25	3.89
	5T	Prop.	**265**	67	**2583**	**48**	**4.6**	**2.17**
		Base.	18	**44**	136	0	30.3	18.3
	10T	Prop.	**281**	54	**2756**	**39**	**3.9**	**1.45**
		Base.	11	**35**	75	0	30.3	21.2

curves (Fig. 3), the blue curve denotes the proposed approach and the red curve the baseline; each curve reports the mean cumulative reward over seven independent runs across 10 million training steps, with the shaded region indicating one standard deviation. Final reward statistics are summarized in Table II, and evaluation metrics measured over 1000 steps are reported in Table IV.

A. Heterogeneous Task Learning

1) Training with the Default vs. Priority-Aware Grid Sensor: The proposed sensor outperforms the baseline in every configuration, with the gap widening as N grows: 812.4 ± 8.7 vs. 558.6 ± 11.3 at 3 types (45.4% improvement), 590.1 vs. 137.7 at 5 types (328.5%), and 445.1 vs. near-zero at 10 types, where the baseline fails to learn (Figs. 3a–3c, Table II).

2) Performance Evaluation: The proposed sensor completes more tasks with fewer incorrect interactions at every configuration (815 vs. 374 at 3 types; 576 vs. 152 at 5; 402 vs. 1 at 10; Table IV). The 10-type baseline's lower incorrect count (8 vs. 203) is misleading, it registered just 1 correct interaction overall. Its marginally lower interaction delay at 3–5 types (1.12, 1.51 vs. 1.51, 1.64 steps) likewise reflects faster but far less accurate task selection, not better performance.

B. Sequential Task Coordination and Generalization

1) Training with Default v.s. Priority-Aware Grid Sensor: The proposed sensor again outperforms the baseline: 720.2 ± 10.4 vs. 218.3 ± 43.6 at 3 types (229.8% improvement), and sustained strong performance at 5 and 10 types (521.3, 366.1) where the baseline collapses to negative reward (Figs. 3d–3f, Table II), indicating it captures the task-state information needed for sequential ordering that the default sensor cannot represent.

2) Performance Evaluation: The same trend holds across every evaluation metric: the proposed method completes 255 sequential chains at 3 types versus 64 for the baseline, and 108 and 31 at 5 and 10 types, where the baseline completes none, while more than halving interaction delay throughout (Table IV).

TABLE V. COLLABORATIVE GENERALIZATION: TRAINED VS. UNSEEN REQUIREMENT VECTORS

Requirement $\mathbf{c}$	$\sum c_j$	Corr.	Incorr.	Acc. (%)	Compl.	Cum. Rew.
$[2,1,0]$, $[0,1,2]$ (trained)	3	358	60	85.6	116	3500
$[1,2,1]$	4	268	63	81.0	66	2617
$[2,2,2]$	6	312	50	86.2	50	3070
$[1,3,2]$	6	226	46	83.1	36	2214

TABLE VI. SEQUENTIAL GENERALIZATION: TRAINED VS. UNSEEN SEQUENCES

Sequence	Len	Corr.	Incorr.	Acc. (%)	Compl.	Cum. Rew.
$T_A \to T_B \to T_C$ (trained)	3	806	330	71.0	255	7730
$T_C \to T_B \to T_A$	3	763	299	71.8	240	7331
$T_B \to T_A \to T_C \to T_B$	4	731	286	71.9	164	7024
$T_A \to T_C \to T_B \to T_A \to T_C$	5	777	313	71.3	137	7457

3) Generalization to Unseen Sequence Orders and Lengths: The generalized formulation teaches per-step recognize-and-dispatch rather than memorized order. The 3-type policy, trained only on $\langle T_A, T_B, T_C \rangle$, evaluated on unseen sequences of different order/length without retraining (Table VI), maintains essentially constant accuracy (71.0–71.9%). Completed chains decrease with length (255, 240, 164, 137) as expected. Stable accuracy confirms generalization.

C. Collaborative Task Coordination and Generalization

1) Training with the Default vs. Priority-Aware Grid Sensor: The proposed approach consistently achieves higher reward: 413.2 ± 4.9 vs. 79.7 ± 10.8 at 3 types (418.3% improvement), sustaining strong performance at 5 and 10 types (205.6, 273.5) against near-zero baseline reward (Figs. 3g–3i, Table II), richer task-requirement information enabling coordination the baseline cannot learn.

2) Performance Evaluation: The proposed approach leads on every metric (Table IV): 116 completed tasks at 3 types vs. 33 for the baseline, and 48 and 39 at 5 and 10 types, where the baseline completes none, with substantially lower interaction and coordination delays (3.9, 1.45 vs. 30.3, 21.2 steps at 10 types), improving both coordination efficiency and effectiveness.

3) Generalization to Unseen Collaborative Requirements: Because each task carries a dynamic requirement vector, a single policy should satisfy vectors never seen in training. We evaluated the 3-type policy, trained only on $\mathbf{c} = [2,1,0]$ and $[0,1,2]$, on unseen vectors $[1,2,1]$, $[2,2,2]$, and $[1,3,2]$ (Table V). Accuracy stays high and stable (81.0–86.2% vs. 85.6% trained). Completions fall as demand $\sum_j c_j$ grows (116, 66, 50, 36), since assembling more robots takes longer; $[1,3,2]$ completes slightly fewer tasks than $[2,2,2]$ at equal demand, reflecting the difficulty of gathering three same-type robots, a case the fixed-composition framework of [6] could not even express.

VI. CONCLUSION AND FUTURE WORK

We presented a generalized MARL framework for heterogeneous multi-robot coordination centered on a priority-aware grid sensor that augments per-cell occupancy with a distance-weighted relevance signal. We generalized sequential tasks to arbitrary-length, arbitrary-order sequences and collaborative tasks to arbitrary requirement vectors with unordered arrival. Evaluated on 3-, 5-, and 10-type configurations against a default-sensor baseline, the perception module substantially improves scalability. Future work includes configurations beyond ten types and real-world deployment, addressing sensor noise, actuation uncertainty, and communication delays.

REFERENCES

[1] C. Yu, A. Velu, E. Vinitsky, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of ppo in cooperative multi-agent games," *NeurIPS Datasets and Benchmarks*, 2022.

[2] K. Low *et al.*, "A survey of multi-agent reinforcement learning for multi-robot systems," *Robotics and Autonomous Systems*, 2024.

[3] M. Bettini *et al.*, "Heterogeneous multi-robot reinforcement learning with shared policies," *Robotics and Autonomous Systems*, 2024.

[4] R. Zhong *et al.*, "Heterogeneous-agent reinforcement learning in markov games," *Artificial Intelligence*, 2024.

[5] J. Yang *et al.*, "Transfer learning for heterogeneous multi-agent systems," *Information Sciences*, 2023.

[6] R. U. Shawon, S. Liu, R. Cox, J. Davis, M. Walters, and R. Powers, "Heterogeneous robots cooperation via multi-agent reinforcement learning," *International Journal of Mechanical Engineering and Robotics Research*, vol. 15, no. 3, 2026.

[7] H. Kono, A. Kamimura, and K. Tomita, "Transfer learning using ontology for heterogeneous multi-agent systems," in *International Conference on Knowledge-Based and Intelligent Information and Engineering Systems*, 2014.

[8] V. Kurin, J. Luketina, G. Farquhar, and J. Foerster, "Shared-policy gradients for multi-agent reinforcement learning," in *International Conference on Learning Representations (ICLR)*, 2021.

[9] J. K. Gupta, M. Egorov, and M. Kochenderfer, "Cooperative multi-agent control using deep reinforcement learning," in *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2017.

[10] T. Wang, T. Gupta, A. Mahajan, B. Peng, S. Whiteson, and C. Zhang, "Rode: Learning roles to decompose multi-agent tasks," in *International Conference on Learning Representations (ICLR)*, 2021.

[11] A. Juliani, V.-P. Berges, E. Vckay, Y. Gao, H. Henry, M. Mattar, and D. Lange, "Unity: A general platform for intelligent agents," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI) Demonstrations Track*, 2018. [Online]. Available: https://arxiv.org/abs/1809.02627

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[13] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

[14] Q. Li, F. Gama, A. Ribeiro, and A. Prorok, "Graph neural networks for decentralized multi-robot path planning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.

[15] A. Siddiqua, S. Liu, K. Mouser, and M. Harris, "Awareness map enabled semi-centralized marl for multi-robot collaboration," in *2025 IEEE 22nd Consumer Communications & Networking Conference (CCNC)*, 2025, pp. 1–6.

[16] R. U. Shawon, S. Liu, and A. Siddiqua, "Explainable reinforcement learning for multi-agent systems," in *2025 IEEE 37th International Conference on Tools with Artificial Intelligence (ICTAI)*. IEEE, 2025, pp. 1339–1343.

[17] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *AAAI Conference on Artificial Intelligence*, 2018.

[18] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Qmix: Monotonic value function factorization for deep multi-agent rl," in *AAAI Conference on Artificial Intelligence*, 2018.

[19] G. Sartoretti, J. Kerr, Y. Shi, G. Wagner, T. K. Satish Kumar, S. Koenig, and H. Choset, "Primal: Pathfinding via reinforcement and imitation multi-agent learning," vol. 4, no. 3. IEEE, 2019, pp. 2378–2385.

[20] M. E. Taylor and P. Stone, "Transfer learning for reinforcement learning domains: A survey," *Journal of Machine Learning Research*, vol. 10, pp. 1633–1685, 2009.